

Supplementary Material for: Do We Really Need All Those Pixels? Deep Learning-Based Task-Oriented Image Communication with JSCC

U. N. Kar, N. Ravi, S. Mukherjee, A. Mishra, T. Srivastava
Vellore Institute of Technology, Andhra Pradesh, India
nishchalravi05@gmail.com

S1. FULL ALGORITHM PSEUDO-CODE

This section translates the three systems described in the main paper into pseudo-code. It is complementary to the paper and aims to make the experiments reproducible. All results in the paper and in this document are on the validation split; the test split has not been used. The code is at <https://github.com/nxck2005/capstone>, and Table S1 lists where each algorithm is implemented.

Notation follows the main paper. k is the number of complex channel uses per image and c the number of complex latent channels, with $c = 8$ at $r = 1/6$ ($k = 12,800$) and $c = 2$ at $r = 1/24$ ($k = 3200$). s is the SNR E_s/N_0 in dB and GN is group normalization with 8 groups. All experiment constants are read from a single parameter file, `spec/params.generated.yaml`; none of the code hard-codes an experiment setting.

A. Deep JSCC

Algorithm S1 gives the DJSCC forward pass. Every system draws its channel noise from `KeyedNoise`(ν, k), which seeds a counter-based (Philox) generator from a noise identity ν alone, draws $\mathbf{a}, \mathbf{b} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_k)$, and returns $(\mathbf{a} + i\mathbf{b})/\sqrt{2}$. At evaluation ν is a hash of the dataset version, the split manifest, the image's stable identifier, the bandwidth ratio, the SNR, the channel seed, and k . It does not depend on the system, so every system at the same bandwidth ratio sees the same noise vector for the same image and SNR, regardless of batching or evaluation order. During training ν also includes the training seed and the epoch, so each image receives fresh noise every epoch.

Algorithm S2 is one training epoch. The fixed-SNR and SNR-randomized models differ only in how s_n is chosen. The PAPR-capped model is trained with the same algorithm and a cap $A = 3$ dB in Algorithm S1.

Algorithm S3 clips symbols whose scaled power would exceed the cap and scales all others up by a common factor until the mean power returns to one, without changing any phase. A solution exists for any cap of at least 0 dB. After projection the code asserts that the mean power is within 10^{-6} of one and that the PAPR is within 10^{-4} dB of the cap.

ALGORITHM S1: DJSCC forward pass

input: image $\mathbf{x} \in [0, 1]^{3 \times 160 \times 160}$, SNR s , noise identity ν , optional PAPR cap A
output: symbols $\mathbf{z} \in \mathbb{C}^k$, reconstruction $\hat{\mathbf{x}}$, class scores $\mathbf{s}$

Encoder
var $\mathbf{h} \leftarrow (\mathbf{x} - \boldsymbol{\mu})/\boldsymbol{\sigma}$
 $\mathbf{h} \leftarrow \text{PReLU}(\text{GN}(\text{Conv}_{5 \times 5/2}^{64}(\mathbf{h})))$ $64 \times 80 \times 80$
 $\mathbf{h} \leftarrow \text{PReLU}(\text{GN}(\text{Conv}_{5 \times 5/2}^{128}(\mathbf{h})))$ $128 \times 40 \times 40$
for $b \leftarrow 1$ **to** 2 **do**
 var $\mathbf{t} \leftarrow \text{PReLU}(\text{GN}(\text{Conv}_{3 \times 3}(\mathbf{h})))$
 $\mathbf{h} \leftarrow \text{PReLU}(\mathbf{h} + \text{GN}(\text{Conv}_{3 \times 3}(\mathbf{t})))$ *residual block*
end
var $\mathbf{p} \leftarrow \text{Conv}_{3 \times 3}^{2c}(\mathbf{h})$ $2c \times 40 \times 40$
Pack channel pairs into complex symbols, channel-major
var $\mathbf{z} \leftarrow \text{flatten}(\mathbf{p}_{0::2} + i\mathbf{p}_{1::2})$ $k = 1600c$
 $\mathbf{z} \leftarrow \mathbf{z} / \sqrt{\frac{1}{k} \sum_i |z_i|^2}$ *unit power per image*
if A is set **then** $\mathbf{z} \leftarrow \text{PeakProject}(\mathbf{z}, A)$
Channel
var $\mathbf{r} \leftarrow \mathbf{z} + 10^{-s/20} \cdot \text{KeyedNoise}(\nu, k)$
Decoder
var $\mathbf{g} \leftarrow \text{PReLU}(\text{GN}(\text{Conv}_{3 \times 3}^{128}(\text{unpack}(\mathbf{r}))))$
for $b \leftarrow 1$ **to** 2 **do** apply a residual block to $\mathbf{g}$ as above
var $\hat{\mathbf{x}} \leftarrow \sigma(\text{Conv}_{4 \times 4/2}^3(\text{PReLU}(\text{GN}(\text{Conv}_{4 \times 4/2}^{64}(\mathbf{g}))))$
var $\mathbf{s} \leftarrow \mathbf{W} \text{GAP}(\mathbf{g}) + \mathbf{b}$ *one linear layer*
return $\mathbf{z}, \hat{\mathbf{x}}, \mathbf{s}$

B. Separated digital baseline

Algorithm S4 finds the transport-block payload for one operating point, and Algorithm S5 runs one image through the chain. A delivered image is classified by the fine-tuned ResNet-18 (the clean one is kept as a secondary stream). Every other outcome is scored with the outage rule, which predicts class 0 and is correct on exactly 100 of the 1000 class-balanced validation images.

Algorithm S6 chooses the operating point for each SNR. It was run twice. The first pass used the clean classifier's codec accuracy. The second, which produced the operating points used in the paper, used the fine-tuned classifier's accuracy and left everything else unchanged. The selected point is then re-run image by image through Algorithm S5, and that realized outcome is what the paper reports.

ALGORITHM S2: One DJSCC training epoch

input: model, epoch e of $E = 100$, weight $\lambda = 3$, base rate $\eta_0 = 10^{-3}$
 $\eta_e \leftarrow \frac{1}{2}\eta_0 (1 + \cos(\pi e/(E-1)))$ *set at epoch start*
order the training set by a permutation keyed on (seed, e)
foreach batch $\{(\mathbf{x}_n, y_n)\}$ of 32 images **do**
 $\mathbf{x}_n \leftarrow$ random resized crop and horizontal flip of $\mathbf{x}_n$
 $s_n \leftarrow 7$ dB, **or** $s_n \sim \text{Uniform}\{1, 4, 7, 13, 19\}$ dB keyed per image and epoch
 $(\mathbf{z}_n, \hat{\mathbf{x}}_n, s_n) \leftarrow$ Algorithm S1 under float16 autocast
 $\mathcal{L} \leftarrow \text{mean}_n [\text{CE}(s_n, y_n) + \lambda \text{MSE}(\hat{\mathbf{x}}_n, \mathbf{x}_n)]$
 scaled backward pass; Adam step at rate η_e , skipped if a gradient overflows
end
evaluate validation top-1 accuracy at 7 dB with keyed noise
keep the checkpoint if it is the best so far (earliest epoch on ties)

ALGORITHM S3: Peak projection

function PeakProject ($\mathbf{z} \in \mathbb{C}^k, A$)
 var $P_i \leftarrow |z_i|^2$; $C \leftarrow 10^{A/10}$
 var $\ell \leftarrow 0$; $u \leftarrow 1$
 Find an upper bracket for the power scale
 for $t \leftarrow 1$ **to** 64 **do**
 if $\frac{1}{k} \sum_i \min(uP_i, C) < 1$ **then** $u \leftarrow 2u$
 end
 Bisect on the common power scale
 for $t \leftarrow 1$ **to** 64 **do**
 var $m \leftarrow (\ell + u)/2$
 if $\frac{1}{k} \sum_i \min(mP_i, C) < 1$ **then** $\ell \leftarrow m$ **else** $u \leftarrow m$
 end
 u is computed from detached powers
 return $z_i \sqrt{\min(uP_i, C)/P_i}$ for all i

C. Task-aware digital control

Algorithm S7 covers the task-aware digital system. Because the digital link either delivers the indices exactly or fails, the model is trained without a channel: its received features equal the transmitted ones whenever a prediction is made. This is also why its accuracy is a constant 82.0% wherever packets arrive (Table S6). The label-transmission system uses the same trained model: the transmitter computes $\hat{y}$ locally, writes it into the low four bits of one byte, zero-pads that byte to the same payload A , and sends it through the same packet, so it shares the same decoding threshold.

S2. CORE CODE

Listings S1–S4 are the actual implementations of the most important components, with argument validation and error handling removed for length. Names and arithmetic are unchanged.

ALGORITHM S4: Transport-block payload

input: channel uses k , bits per symbol Q_m , nominal LDPC rate R
output: payload size A in bits, or infeasible
var $G \leftarrow kQ_m$; $N \leftarrow \lfloor GR \rfloor$
Largest byte-aligned payload whose packetization fits
for $A \leftarrow 8\lfloor N/8 \rfloor$ **down to** 8 **step** 8 **do**
 segment A per TS 38.212: TB CRC, base graph, code blocks with CB CRC, Z, K'
 distribute G over the code blocks as budgets $E_1, \dots, E_C$
 if $K'/\min_r E_r \leq 0.95$ **and** $A + |\text{CRC}_{\text{TB}}| \leq N$ **then break**
end
if no A qualified **then return infeasible**
Respect the base graph's minimum code rate
while $K'/\max_r E_r < R_{\min}(\text{BG})$ **do** $A \leftarrow A + 8$ and re-plan
return A

ALGORITHM S5: Classical chain for one image

input: image $\mathbf{x}$, encode size a , LDPC rate R , modulation, SNR s , noise identity ν
output: structural infeasibility, codec infeasibility, decode failure, or delivered $\hat{\mathbf{x}}$
 $A \leftarrow$ Algorithm S4
if infeasible then return structural infeasibility
Source coding
 $\mathbf{x}_a \leftarrow$ bilinear downsample of $\mathbf{x}$ to $a \times a$
bisect the JPEG 2000 compression ratio for the largest codestream of at most $A/8$ bytes
if no codestream fits then return codec infeasibility
zero-pad the codestream to exactly $A/8$ bytes
Channel coding and modulation
TB CRC; segmentation and CB CRC; LDPC encoding; rate matching with filler
bit interleaving; Gray-mapped BPSK, QPSK, or 16-QAM at unit average energy
add $10^{-s/20} \cdot \text{KeyedNoise}(\nu, k)$
max-log LLRs; offset min-sum decoding (offset 0.5, 50 iterations)
if any CRC fails then return decode failure
Source decoding
strip filler; decode the codestream; bicubic upsample to 160×160
return delivered, $\hat{\mathbf{x}}$

Listing S1. DJSCC residual block, complex packing, and encoder forward pass (src/models/djscc.py).

```

class ResidualBlock(nn.Module):
    def __init__(self, width):
        super().__init__()
        self.conv1 = nn.Conv2d(width, width, kernel_size=3, padding=1)
        self.norm1 = nn.GroupNorm(8, width)
        self.activation1 = nn.PReLU(width)
        self.conv2 = nn.Conv2d(width, width, kernel_size=3, padding=1)
        self.norm2 = nn.GroupNorm(8, width)
        self.activation2 = nn.PReLU(width)

    def forward(self, inputs):
        residual = self.activation1(self.norm1(self.conv1(inputs)))
        residual = self.norm2(self.conv2(residual))
        return self.activation2(inputs + residual)

def pack_complex_symbols(projection):
    """Pack adjacent real/imaginary channels, then flatten C, row, column."""
    batch, doubled_channels, height, width = projection.shape
    paired = projection.reshape(batch, doubled_channels // 2, 2, height, width)
    if paired.dtype in (torch.float16, torch.bfloat16):
        paired = paired.float() # keep the channel interface at complex64 under AMP
    symbols = torch.complex(paired[:, :, 0], paired[:, :, 1])
    return symbols.flatten(start_dim=1)

class DJSCCEncoder(nn.Module):
    def forward(self, inputs):
        features = self.residual(self.body_entry(self.stem(self.input_normalisation(inputs))))
        symbols = pack_complex_symbols(self.projection(features))
        symbols = normalize_unit_average_power(symbols)
        if self.peak_constraint is not None:
            symbols = self.peak_constraint(symbols)
        return symbols

```

ALGORITHM S6: Per-SNR operating-point selection

input: SNR s ; candidates $\mathcal{C}$ of (encode size, R , modulation);
 measured BLER curves; measured codec accuracy $A_{\text{codec}}(c)$;
 outage accuracy $A_{\text{out}} = 0.100$

output: operating point c^*

foreach feasible $c \in \mathcal{C}$ **do**

foreach code block r of c 's packet **do**

$\text{BLER}_r(s) \leftarrow$ measured curve, linear in $\log_{10}$ BLER
 between measured SNRs

if s is outside the measured range **then** exclude c

end

$P_s(c) \leftarrow \prod_r (1 - \text{BLER}_r(s))$

$E(c) \leftarrow P_s(c) A_{\text{codec}}(c) + (1 - P_s(c)) A_{\text{out}}$

end

Exact ties: higher P_s , lower Q_m , lower R , larger image, then identifier

return $\arg \max_c E(c)$

ALGORITHM S7: Task-aware digital system

input: $D = 2048$, $b = 2$ bits, $L = 4$ levels, step $\Delta = 2/(L - 1)$

Encoder

var $\mathbf{h} \leftarrow$ DJSCC encoder trunk (Algorithm S1, up to the residual blocks)

var $\mathbf{u} \leftarrow \text{AvgPool}_{8 \times 8}(\text{Conv}_{3 \times 3}^{D/64}(\mathbf{h}))$ $32 \times 8 \times 8$

var $\mathbf{q} \leftarrow \text{clamp}(\text{round}((\tanh \mathbf{u} + 1)/\Delta), 0, L - 1)$

Training: no channel, straight-through estimator

$\tilde{\mathbf{u}} \leftarrow \tanh \mathbf{u} + \text{sg}(\mathbf{q}\Delta - 1 - \tanh \mathbf{u})$

$\mathcal{L} \leftarrow \text{CE}(\mathbf{W} \text{ GAP}(\tilde{\mathbf{u}}) + \mathbf{b}, \mathbf{y})$

Transmission

var $\mathbf{m}_{\text{rc}} \leftarrow$ range-code $\mathbf{q}$ with a static model fitted on training indices

var $\mathbf{m}_{\text{raw}} \leftarrow \mathbf{q}$ as $bD = 4096$ raw bits

if $|\mathbf{m}_{\text{rc}}| \leq |\mathbf{m}_{\text{raw}}|$ **then** $\mathbf{m} \leftarrow [0, \mathbf{m}_{\text{rc}}]$ **else** $\mathbf{m} \leftarrow [1, \mathbf{m}_{\text{raw}}]$

send $\mathbf{m}$, zero-padded to A , through the channel-coding steps of Algorithm S5 (BPSK, rate 1/3)

Reception

if a CRC fails **then** predict the outage class **else**

$\hat{\mathbf{y}} \leftarrow \arg \max(\mathbf{W} \text{ GAP}(\mathbf{q}\Delta - 1) + \mathbf{b})$

Listing S2. Power normalization, symbol-domain PAPR, and the peak projection (src/channels/power.py).

```

def normalize_unit_average_power(symbols):
    """Normalise every image independently to mean_k  $|x|^2 = 1$ ."""
    powers = symbols.abs().square()
    return symbols * torch.rsqrt(powers.mean(dim=1)).unsqueeze(1)

def symbol_papr_db(symbols):
    """Symbol-domain PAPR in dB per image (not oversampled waveform PAPR)."""
    powers = symbols.abs().square()
    return 10.0 * torch.log10(powers.amax(dim=1) / powers.mean(dim=1))

class PeakPowerConstraint(nn.Module):
    def forward(self, symbols):
        powers = symbols.abs().square()
        detached = powers.detach().to(torch.float64)
        cap = torch.as_tensor(self.max_power, dtype=torch.float64, device=symbols.device)
        low = torch.zeros((symbols.shape[0], 1), dtype=torch.float64, device=symbols.device)
        high = torch.ones_like(low)
        for _ in range(64):
            # bracket
            achieved = torch.minimum(high * detached, cap).mean(dim=1, keepdim=True)
            high = torch.where(achieved < 1, high * 2, high)
        for _ in range(64):
            # bisection
            midpoint = (low + high) / 2
            achieved = torch.minimum(midpoint * detached, cap).mean(dim=1, keepdim=True)
            low = torch.where(achieved < 1, midpoint, low)
            high = torch.where(achieved >= 1, midpoint, high)
        scale = high.to(powers.dtype)
        projected_powers = torch.minimum(scale * powers, powers.new_tensor(self.max_power))
        support = powers > 0
        amplitude_scale = torch.zeros_like(projected_powers)
        amplitude_scale[support] = torch.sqrt(projected_powers[support] / powers[support])
        return symbols * amplitude_scale

```

TABLE S1
WHERE EACH COMPONENT IS IMPLEMENTED.

Component	File
DJSCC model (Alg. S1)	src/models/djscc.py
Power, PAPR, peak projection (Alg. S3)	src/channels/power.py
Keyed AWGN	src/channels/awgn.py
Training objective	src/training/djscc_loss.py
Training loops (Alg. S2)	src/training/w8_final.py, er2_randomized.py, papr_constrained.py
Payload solver (Alg. S4)	src/baseline/ldpc/transport. py
JPEG 2000 rate control	src/baseline/j2k.py
Classical chain (Alg. S5)	src/baseline/classical/ pipeline.py
Selection (Alg. S6)	src/baseline/classical/ composition.py
Task-aware model (Alg. S7)	src/models/er9_digital.py
Quantizer, message format	src/evaluation/er9_protocol. py
Reference classifiers	src/models/reference_ classifier.py
Validation evaluation	src/evaluation/w10_backends. py

Listing S3. Keyed complex noise, the AWGN channel, and the training objective (src/channels/awgn.py, src/training/djscc_loss.py).

```
def keyed_complex_noise(noise_ids, k, *, dtype=torch.complex64, device=None):
    """Unit-variance complex noise as a pure function of each noise_id."""
    rows = []
    for noise_id in noise_ids:
        components = keyed_standard_normal("channel_noise", {"noise_id": noise_id}, size=(2, k))
        rows.append((components[0] + 1j * components[1]) * math.sqrt(0.5))
    return torch.as_tensor(np.stack(rows), dtype=dtype, device=device)

class AWGN(nn.Module):
    """Complex AWGN with  $E|n|^2 = 1/\text{gamma}$  after unit-power normalisation."""
    def forward(self, symbols, snr_db, *, unit_noise=None):
        snr = torch.as_tensor(snr_db, dtype=symbols.real.dtype, device=symbols.device)
        if snr.ndim == 0:
            snr = snr.expand(symbols.shape[0]) # scalar or one SNR per sample
        if unit_noise is None:
            unit_noise = torch.complex(torch.randn_like(symbols.real),
                                       torch.randn_like(symbols.real)) * math.sqrt(0.5)
        gamma = torch.pow(torch.as_tensor(10.0, device=symbols.device), snr / 10.0)
        return symbols + unit_noise * torch.rsqrt(gamma).unsqueeze(1)

class DJSCCObjective(nn.Module):
    """CE + lambda * MSE, with lambda retained as data, including zero."""
    def forward(self, output, targets, augmented_inputs):
        cross_entropy = F.cross_entropy(output.logits, targets)
        reconstruction_mse = F.mse_loss(output.reconstruction, augmented_inputs)
        total = cross_entropy + self.reconstruction_weight * reconstruction_mse
        return DJSCCLoss(total=total, cross_entropy=cross_entropy, reconstruction_mse=reconstruction_mse)
```

Listing S4. Operating-point composition and BLER interpolation (src/baseline/classical/composition.py), and the task-aware quantizer (src/evaluation/er9_protocol.py).

```
def success_probability(block_blers):
    """P(TB success) = product over code blocks of (1 - BLER_r)."""
    probability = 1.0
    for bler in block_blers:
        probability *= 1.0 - bler
    return probability

def expected_accuracy(*, success_probability, codec_accuracy, outage_accuracy):
    # Both accuracy terms are measured counts on the validation split, never 1/n_classes.
    return success_probability * codec_accuracy.value + (1.0 - success_probability) * outage_accuracy.value

def _interpolate(curve, snr):
    """Linear in SNR against log10(BLER), strictly inside the measured span."""
    for point_snr, point_bler in zip(curve.snr_db, curve.bler):
        if point_snr == snr:
            return point_bler
    for i in range(len(curve.snr_db) - 1):
        low, high = curve.snr_db[i], curve.snr_db[i + 1]
        if low < snr < high:
            w = (snr - low) / (high - low)
            return 10 ** ((1 - w) * math.log10(curve.bler[i]) + w * math.log10(curve.bler[i + 1]))
    raise UncharacterizedBlerError(snr) # the candidate is excluded, not given a default

class UniformScalarQuantizer:
    def __init__(self, bits):
        self.levels = 2 ** bits
        self.step = 2.0 / (self.levels - 1) # fixed [-1, +1] range, no side information

    def quantise(self, features):
        scaled = (torch.tanh(features) + 1.0) / self.step
        return torch.round(scaled).clamp(0, self.levels - 1).to(torch.long)

    def dequantise(self, indices):
        return indices.to(torch.float32) * self.step - 1.0

    def straight_through(self, features):
        bounded = torch.tanh(features)
        indices = self.quantise(features)
        dequantised = self.dequantise(indices).to(features.dtype)
        return bounded + (dequantised - bounded).detach(), indices
```

S3. ARCHITECTURE AND TRAINING SETTINGS

Table S2 gives the model sizes and Table S3 the training and link settings.

TABLE S2
MODEL SIZES (PARAMETERS).

Model	Encoder	Decoder	Total
DJSCC, $r = 1/6$	820,688	746,509	1,567,197
DJSCC, $r = 1/24$	806,852	732,685	1,539,537
DJSCC, $r = 1/2$ (profile)	857,584	783,373	1,640,957
ResNet-18 classifier	—	—	11,181,642

TABLE S3
TRAINING AND DIGITAL-LINK SETTINGS.

Setting	Value
Optimizer	Adam, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, no weight decay
Learning rate	10^{-3} , cosine to 0 over 100 epochs, no warm-up
Batch	32 images, last partial batch kept
Augmentation	random resized crop, horizontal flip
Precision	float16 autocast, dynamic gradient scaling
Training SNR	7 dB, or $\{1, 4, 7, 13, 19\}$ dB per image and epoch
Loss	CE + $3 \cdot \text{MSE}$ (DJSCC); CE (task-aware digital)
Selection	best validation top-1 at 7 dB, earliest on ties
Seeds	three paired seeds per DJSCC ratio and for the task-aware model; one each for the randomized and PAPR-capped models; results use seed pair 0
Fine-tuned classifier	from the clean ResNet-18, 20 epochs (5 warm-up) on 44,039 JPEG 2000-decoded training images
Channel code	5G NR LDPC per TS 38.212 V17.13.0; Sionna 2.0.1 encoder and decoder
LDPC rates	1/3, 1/2, 2/3, 5/6
Decoder	offset min-sum, offset 0.5, 50 iterations, flooding, LLR clip 20
Modulation	BPSK, QPSK, 16-QAM; Gray mapping; TS 38.212 bit interleaver; max-log demapping
Source codec	OpenJPEG 2.5.4: raw codestream, 9/7 wavelet, 6 levels, RPCL, 64×64 blocks
Rate control	bisection on compression ratio (16-byte tolerance, at most 24 encodes)
Resizing	bilinear down, bicubic up

S4. VALIDATION SELECTION RECORDS

Table S4 records the choice of the reconstruction weight λ at $r = 1/6$. The rule admits every λ within one point of the $\lambda = 0$ accuracy (floor 81.7%), keeps those whose mean reconstruction PSNR at 15 dB is at least 20 dB (falling back to 16 dB if none qualifies), and takes the smallest λ remaining. Only $\lambda = 3$ reached 20 dB. Table S5 records the choice of the task-aware feature dimension, measured through the real digital chain at 7 dB.

TABLE S4
CHOICE OF λ . ACCURACY IS VALIDATION TOP-1 AT 7 dB; PSNR IS AT 15 dB.

λ	Epoch	Acc. (%)	PSNR (dB)	≥ 20 dB
0	84	82.7	9.7	
0.1	75	83.5	17.6	
0.3	71	82.2	18.7	
1	67	82.8	19.9	
3	92	83.2	21.2	✓

TABLE S5
CHOICE OF THE TASK-AWARE FEATURE DIMENSION D ($b = 2$).

D	Raw payload (bits)	Acc. (%)
64	128	28.4
128	256	55.2
256	512	75.9
512	1024	79.3
1024	2048	81.8
2048	4096	82.0

S5. COMPLETE VALIDATION RESULTS

Tables S6–S11 give every published validation measurement. Table IV of the main paper is a subset of Table S6, and its Table V is the $r = 1/6$ counterpart of Table S9.

The symbol-domain PAPR over the 1000 validation images is: DJSCC at $r = 1/6$, mean 13.22 dB and maximum 20.06 dB; DJSCC at $r = 1/24$, mean 12.21 dB and maximum 19.48 dB; SNR-randomized DJSCC, mean 13.26 dB and maximum 20.72 dB; PAPR-capped DJSCC, mean 3.000001 dB and maximum 3.000002 dB.

TABLE S6

VALIDATION TOP-1 ACCURACY (%) OF EVERY SYSTEM VARIANT AT EVERY MEASURED SNR (1000 IMAGES PER ENTRY, PRIMARY SCORER).

System	SNR E_s/N_0 (dB)																				
	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7	9	11	13	15	18
DJSCC	72.8	73.8	75.7	77.5	79.4	79.9	80.7	81.5	81.2	82.6	82.0	82.6	82.8	83.2	83.2	83.6	83.8	83.4	83.8	83.5	83.4
DJSCC, SNR-rand.	77.0	78.6	80.2	82.4	82.4	82.6	82.5	83.1	83.0	83.7	83.2	83.7	82.9	83.0	83.6	83.8	84.1	83.8	83.8	83.8	83.9
DJSCC, PAPR-capped	75.9	78.7	78.8	80.1	80.5	80.8	80.7	82.2	82.3	83.0	82.6	83.3	82.9	83.7	82.9	82.8	83.0	83.2	83.3	83.5	83.2
DJSCC recon. + clean	44.7	48.9	54.5	55.7	61.1	61.8	63.9	65.8	69.0	70.5	73.5	74.2	74.1	75.9	76.1	76.1	77.5	77.0	77.1	76.8	76.8
J2K adaptive	10.0	10.0	10.0	10.0	83.4	83.4	83.9	87.3	87.3	87.3	87.8	87.8	87.8	87.8	88.7	89.0	86.5	89.3	89.3	89.3	89.3
J2K QPSK only	10.0	10.0	10.0	10.0	10.0	10.0	10.0	87.3	87.3	87.3	87.8	87.8	87.8	87.8	88.7	88.7	88.7	88.7	88.7	88.7	88.7
J2K fixed MCS	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	89.0	89.0	89.0	89.0	89.0	89.0
JPEG adaptive	10.0	10.0	10.0	10.0	67.8	67.8	76.7	84.0	84.0	84.0	85.6	85.6	87.2	87.2	87.9	87.9	10.0	88.6	88.6	88.6	88.6
Task-aware digital	10.0	10.0	10.0	10.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0
Label transmission	10.0	10.0	10.0	10.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0	82.0
DJSCC (1/24)	49.4	56.0	61.7	67.4	69.3	73.6	76.2	78.5	78.7	80.6	80.8	81.8	82.0	82.0	82.5	82.9	82.7	83.5	83.2	83.5	82.8
J2K adaptive (1/24)	10.0	10.0	10.0	10.0	10.0	10.0	31.0	69.4	69.4	75.8	80.6	80.6	83.4	83.4	84.6	85.8	82.4	87.3	87.7	87.7	87.7

TABLE S7

FRACTION OF PACKETS DELIVERED (%) FOR THE DIGITAL SYSTEMS. DJSCC VARIANTS ALWAYS PRODUCE AN OUTPUT AND ARE OMITTED.

System	SNR E_s/N_0 (dB)																				
	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7	9	11	13	15	18
J2K adaptive	0.0	0.0	0.0	0.0	100.0	100.0	97.7	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	96.2	100.0	100.0	100.0	100.0
J2K QPSK only	0.0	0.0	0.0	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
J2K fixed MCS	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0
JPEG adaptive	0.0	0.0	0.0	0.0	100.0	100.0	97.8	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	0.0	100.0	100.0	100.0	100.0
Task-aware digital	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Label transmission	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
J2K adaptive (1/24)	0.0	0.0	0.0	0.0	0.0	0.0	82.4	100.0	100.0	91.5	100.0	100.0	100.0	100.0	100.0	100.0	93.8	100.0	100.0	100.0	100.0

TABLE S8

ACCURACY (%) OF THE SAME DIGITAL OUTPUTS SCORED BY THE FINE-TUNED AND THE CLEAN RESNET-18.

System	SNR E_s/N_0 (dB)																					
	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7	9	11	13	15	18	
J2K adaptive, fine-tuned	10.0	10.0	10.0	10.0	83.4	83.4	83.9	87.3	87.3	87.3	87.8	87.8	87.8	87.8	88.7	89.0	86.5	89.3	89.3	89.3	89.3	
J2K adaptive, clean	10.0	10.0	10.0	10.0	60.9	60.9	78.8	84.8	84.8	84.8	87.0	87.0	87.0	87.0	88.4	88.6	86.3	89.2	89.2	89.2	89.2	
J2K QPSK only, fine-tuned	10.0	10.0	10.0	10.0	10.0	10.0	10.0	87.3	87.3	87.3	87.8	87.8	87.8	87.8	88.7	88.7	88.7	88.7	88.7	88.7	88.7	
J2K QPSK only, clean	10.0	10.0	10.0	10.0	10.0	10.0	10.0	84.8	84.8	84.8	87.0	87.0	87.0	87.0	88.4	88.4	88.4	88.4	88.4	88.4	88.4	
J2K fixed MCS, fine-tuned	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	89.0	89.0	89.0	89.0	89.0	89.0	
J2K fixed MCS, clean	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.0	88.6	88.6	88.6	88.6	88.6	88.6	
JPEG adaptive, fine-tuned	10.0	10.0	10.0	10.0	67.8	67.8	76.7	84.0	84.0	84.0	85.6	85.6	87.2	87.2	87.9	87.9	10.0	88.6	88.6	88.6	88.6	
JPEG adaptive, clean	10.0	10.0	10.0	10.0	45.2	45.2	62.3	71.0	71.0	71.0	83.9	83.9	85.8	85.8	88.2	88.2	10.0	88.9	88.9	88.9	88.9	
J2K adaptive (1/24), fine-tuned	10.0	10.0	10.0	10.0	10.0	10.0	31.0	69.4	69.4	75.8	80.6	80.6	83.4	83.4	84.6	85.8	82.4	87.3	87.7	87.7	87.7	
J2K adaptive (1/24), clean	10.0	10.0	10.0	10.0	10.0	10.0	14.2	26.2	26.2	48.0	50.7	50.7	60.9	60.9	77.4	80.6	80.7	84.8	87.0	87.0	87.0	

TABLE S9
OPERATING POINTS OF THE ADAPTIVE JPEG 2000 BASELINE AT $r = 1/24$.

SNR (dB)	Image (px)	LDPC	Mod.	Acc. (%)	Deliv. (%)
-8	160	1/3	BPSK	10.0	0.0
-7	160	1/3	BPSK	10.0	0.0
-6	160	1/3	BPSK	10.0	0.0
-5	160	1/3	BPSK	10.0	0.0
-4	160	1/3	BPSK	10.0	0.0
-3	160	1/3	BPSK	10.0	0.0
-2	96	1/2	BPSK	31.0	82.4
-1	160	1/3	QPSK	69.4	100.0
0	160	1/3	QPSK	69.4	100.0
1	96	1/2	QPSK	75.8	91.5
2	96	1/2	QPSK	80.6	100.0
3	96	1/2	QPSK	80.6	100.0
4	64	1/3	16-QAM	83.4	100.0
5	64	1/3	16-QAM	83.4	100.0
6	96	5/6	QPSK	84.6	100.0
7	96	1/2	16-QAM	85.8	100.0
9	128	2/3	16-QAM	82.4	93.8
11	128	2/3	16-QAM	87.3	100.0
13	128	5/6	16-QAM	87.7	100.0
15	128	5/6	16-QAM	87.7	100.0
18	128	5/6	16-QAM	87.7	100.0

TABLE S11
OPERATING POINTS OF THE QPSK-ONLY JPEG 2000 BASELINE AT
 $r = 1/6$.

SNR (dB)	Image (px)	LDPC	Mod.	Acc. (%)	Deliv. (%)
-8	160	1/3	QPSK	10.0	0.0
-7	160	1/3	QPSK	10.0	0.0
-6	160	1/3	QPSK	10.0	0.0
-5	160	1/3	QPSK	10.0	0.0
-4	160	1/3	QPSK	10.0	0.0
-3	160	1/3	QPSK	10.0	0.0
-2	160	1/3	QPSK	10.0	0.0
-1	128	1/3	QPSK	87.3	100.0
0	128	1/3	QPSK	87.3	100.0
1	128	1/3	QPSK	87.3	100.0
2	128	1/2	QPSK	87.8	100.0
3	128	1/2	QPSK	87.8	100.0
4	128	1/2	QPSK	87.8	100.0
5	128	1/2	QPSK	87.8	100.0
6	160	5/6	QPSK	88.7	100.0
7	160	5/6	QPSK	88.7	100.0
9	160	5/6	QPSK	88.7	100.0
11	160	5/6	QPSK	88.7	100.0
13	160	5/6	QPSK	88.7	100.0
15	160	5/6	QPSK	88.7	100.0
18	160	5/6	QPSK	88.7	100.0

TABLE S10
OPERATING POINTS OF THE BASELINE JPEG SYSTEM AT $r = 1/6$.

SNR (dB)	Image (px)	JPEG Q	LDPC	Mod.	Acc. (%)	Deliv. (%)
-8	64	5	1/3	BPSK	10.0	0.0
-7	64	5	1/3	BPSK	10.0	0.0
-6	64	5	1/3	BPSK	10.0	0.0
-5	64	5	1/3	BPSK	10.0	0.0
-4	64	5	1/3	BPSK	67.8	100.0
-3	64	5	1/3	BPSK	67.8	100.0
-2	64	10	1/2	BPSK	76.7	97.8
-1	64	20	1/3	QPSK	84.0	100.0
0	64	20	1/3	QPSK	84.0	100.0
1	64	20	2/3	BPSK	84.0	100.0
2	96	15	1/2	QPSK	85.6	100.0
3	96	15	1/2	QPSK	85.6	100.0
4	96	30	2/3	QPSK	87.2	100.0
5	96	30	2/3	QPSK	87.2	100.0
6	128	20	5/6	QPSK	87.9	100.0
7	128	20	5/6	QPSK	87.9	100.0
9	160	15	2/3	16-QAM	10.0	0.0
11	160	15	2/3	16-QAM	88.6	100.0
13	160	15	2/3	16-QAM	88.6	100.0
15	160	15	2/3	16-QAM	88.6	100.0
18	160	15	2/3	16-QAM	88.6	100.0

S6. REPRODUCING THE FIGURES AND TABLES

The paper’s figures and all tables in this document are regenerated from the committed result files, without model weights, datasets, or a GPU:

```
git clone https://github.com/nxck2005/capstone
cd capstone
python3 deliverables/research-paper/figures/make_figures.py
python3 deliverables/research-paper/supplement/make_tables.py
cd deliverables/research-paper
latexmk -pdf capstone_rp.tex
cd supplement && latexmk -pdf capstone_rp_supplement.tex
```

Both scripts read only `presentation-results/data/*.csv` and `results/learned/w10/w10_continuation_units_v11.json`. Re-running training or evaluation requires the Imagenette-160 archive, an NVIDIA GPU, and the environment pinned in `requirements.lock`.

ACKNOWLEDGMENT

This supplementary material was produced with Anthropic’s Claude Opus 5.5. It wrote the pseudo-code in Section S1 from the project’s source code, selected and condensed the code excerpts in Section S2, drafted the text throughout, and generated the tables in Sections S3–S5 with scripts that read the published result files. The code excerpted in Section S2 was written with the assistance of Claude Opus 5.5 and OpenAI’s GPT-5.6 Sol. All content was reviewed and verified by the authors, who take full responsibility for it.